

JOSÉ ANTONIO PASTOR RIVERA - RA3

Parte 1: Diseño de pruebas

Tarea 1: Identificación de pruebas

Para asegurar la calidad de la clase Calculadora, es necesario aplicar una estrategia combinada de pruebas:

- **Pruebas Unitarias (White Box):** Son imprescindibles porque necesitamos verificar la lógica interna de cada método de forma aislada. Al ser una clase de lógica matemática pura, debemos asegurar que las operaciones (+, -, *, /) se realizan correctamente a nivel de código antes de integrar esta clase en sistemas más complejos como el GestorFacturas.
- **Pruebas de Caja Negra (Black Box):** Son necesarias para validar el cumplimiento de los requisitos funcionales sin dejarnos influir por el código escrito. Esto es crucial para detectar errores lógicos como el del método restar, donde el código compila y ejecuta (caja blanca correcta), pero el resultado funcional es incorrecto (resta al revés) respecto a lo que espera el usuario. Además, este enfoque nos permite verificar cómo se comporta el sistema ante **valores límite** (bordes) y entradas no válidas, asegurando la robustez del software.

Tarea 2: Definición de casos de prueba

A continuación se definen los casos de prueba para el método dividir.

Nota: El código proporcionado utiliza tipos double, por lo que los resultados esperados incluyen decimales.

Caso de Prueba	Entradas (a, b)	Resultado esperado	Justificación
CP-DIV-01	(10, 2)	5.0	Caso normal: División exacta estándar.
CP-DIV-02	(5, 2)	2.5 (2 si es división entera)	Precisión: Verificación de decimales
CP-DIV-03	(10, 0)	Error (ArithmeticException)	Caso de error: La división por cero no está definida y debe ser controlada.
CP-DIV-04	(0, 5)	0.0 (límite)	Valor límite: Dividir cero entre un número es una operación válida.
CP-DIV-05	(-10, 2)	-5.0	Ley de signos: Verificación con números negativos.

Nota: Se ha añadido código a CalculadoraTest.java para poder verificar CP-DIV-05.

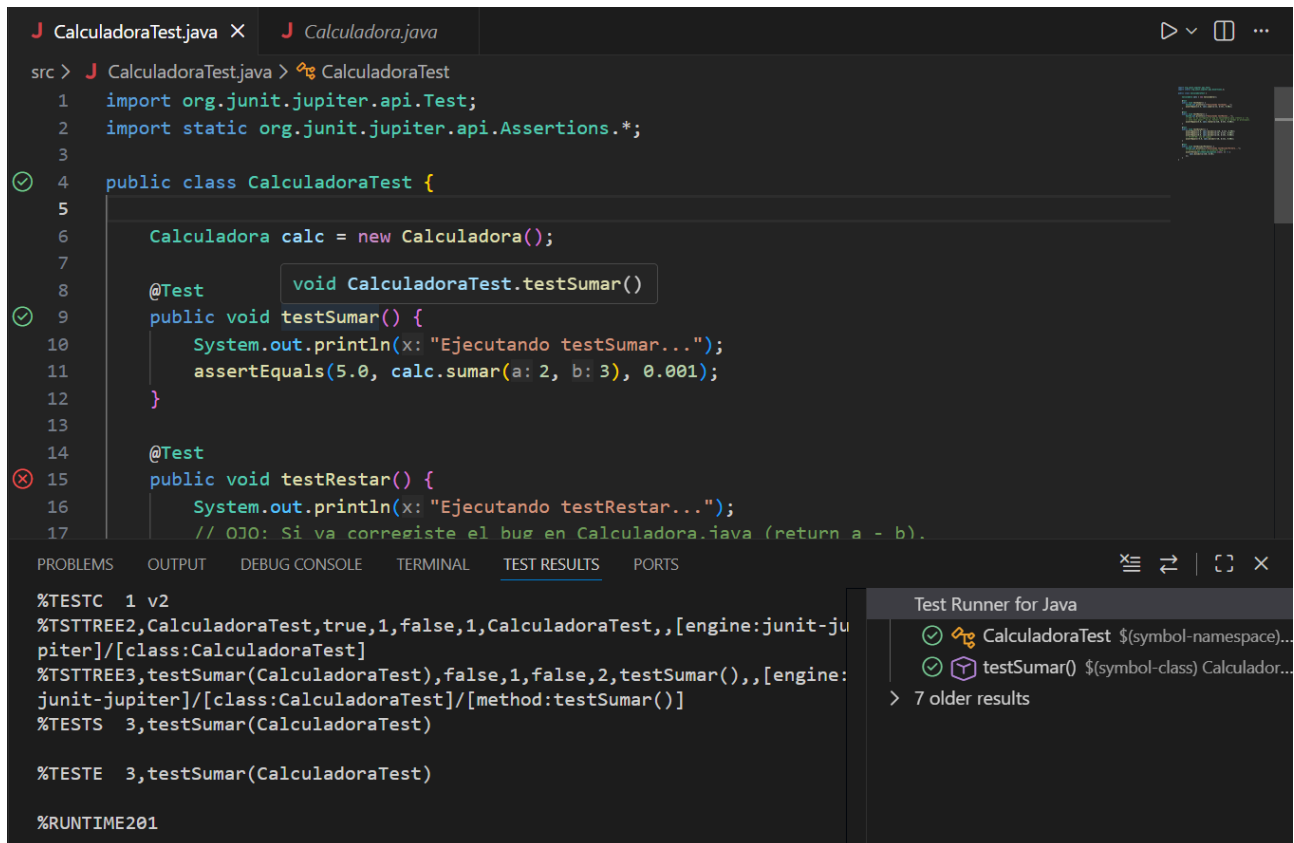
```
@Test
public void testDividir() {
    assertEquals(5.0, calc.dividir(a: 10, b: 2), 0.001);
    assertEquals(2.5, calc.dividir(a: 5, b: 2), 0.001);
    assertEquals(0.0, calc.dividir(a: 0, b: 5), 0.001);
    // CP-DIV-05: Ley de signos
    assertEquals(-5.0, calc.dividir(-10, b: 2), 0.001);
}
```

Parte 2: Pruebas automatizadas con JUnit

Tarea 3: Implementación de pruebas

Se ha creado la clase de pruebas CalculadoraTest.java utilizando el framework JUnit 5 en Visual Studio Code. Se han implementado los siguientes casos de prueba para cubrir los requisitos y verificar los bugs detectados en el diseño:

1. **testSumar:** Verifica el funcionamiento correcto del método sumar.



The screenshot displays the Visual Studio Code interface with two tabs: `CalculadoraTest.java` and `Calculadora.java`. The `CalculadoraTest.java` file contains the following code:

```
src > J CalculadoraTest.java > CalculadoraTest
1  import org.junit.jupiter.api.Test;
2  import static org.junit.jupiter.api.Assertions.*;
3
4  public class CalculadoraTest {
5
6      Calculadora calc = new Calculadora();
7
8      @Test
9      public void testSumar() {
10         System.out.println(x: "Ejecutando testSumar...");
11         assertEquals(5.0, calc.sumar(a: 2, b: 3), 0.001);
12     }
13
14     @Test
15     public void testRestar() {
16         System.out.println(x: "Ejecutando testRestar...");
17         // OJO: Si va corregiste el bug en Calculadora.java (return a - b).
```

The `TEST RESULTS` panel shows the following output:

```
%TESTC 1 v2
%TSTTREE2,CalculadoraTest,true,1,false,1,CalculadoraTest,,[engine:junit-jupiter]/[class:CalculadoraTest]
%TSTTREE3,testSumar(CalculadoraTest),false,1,false,2,testSumar(),,[engine:junit-jupiter]/[class:CalculadoraTest]/[method:testSumar()]
%TESTS 3,testSumar(CalculadoraTest)

%TESTE 3,testSumar(CalculadoraTest)

%RUNTIME201
```

The `Test Runner for Java` panel shows the following results:

- ✓ `CalculadoraTest` \$(symbol-namespace)...
- ✓ `testSumar()` \$(symbol-class) Calculador...

> 7 older results

2. testRestar: Diseño para validar la resta $10 - 5$.

The screenshot shows an IDE with two tabs: `CalculadoraTest.java` and `Calculadora.java`. The `CalculadoraTest` class contains two test methods. The `testRestar()` method is failing, indicated by a red 'X' icon. The error message in the Test Runner for Java panel is: `Expected [5.0] but was [-5.0]` and `org.opentest4j.AssertionFailedError: expected [5.0] but was [-5.0]`. The terminal output shows the test results and the error message.

```
src > J CalculadoraTest.java > CalculadoraTest
4 public class CalculadoraTest {
6     Calculadora calc = new Calculadora();
7
8     @Test
9     public void testSumar() {
10         System.out.println(x: "Ejecutando testSumar...");
11         assertEquals(5.0, calc.sumar(a: 2, b: 3), 0.001);
12     }
13
14     @Test
15     public void testRestar() {
16         System.out.println(x: "Ejecutando testRestar...");
17         // OJO: Si ya corregiste el bug en Calculadora.java (return a - b),
18         // este test pasará. Si no, fallará (que es lo normal al principio).
19         assertEquals(5.0, calc.restar(a: 10, b: 5), 0.001);
20     }
21 }
```

Test Runner for Java

- CalculadoraTest \$(symbol-namespace)...
- testRestar() \$(symbol-class) Calculador...
 - Expected [5.0] but was [-5.0]
 - org.opentest4j.AssertionFailedError: expec...

8 older results

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

at org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:1014)

at CalculadoraTest.testRestar(CalculadoraTest.java:19)

at java.base/java.lang.reflect.Method.invoke(Method.java:565)

at java.base/java.util.ArrayList.forEach(ArrayList.java:1604)

at java.base/java.util.ArrayList.forEach(ArrayList.java:1604)

%TRACEE

%TESTE 3,testRestar(CalculadoraTest)

%RUNTIME153

3. testDividir: Incluye los casos de prueba diseñados en la Tarea 2 (CP-DIV-01, CP-DIV-02, CP-DIV-04 y CP-DIV-05).

The screenshot shows the same IDE with the `CalculadoraTest` class. The `testDividir()` method is passing, indicated by a green checkmark. The Test Runner for Java panel shows the test results and the error message. The terminal output shows the test results and the error message.

```
src > J CalculadoraTest.java > CalculadoraTest
4 public class CalculadoraTest {
21
22     @Test
23     public void testDividir() {
24         assertEquals(5.0, calc.dividir(a: 10, b: 2), 0.001);
25         assertEquals(2.5, calc.dividir(a: 5, b: 2), 0.001);
26         assertEquals(0.0, calc.dividir(a: 0, b: 5), 0.001);
27         // CP-DIV-05: Ley de signos
28         assertEquals(-5.0, calc.dividir(-10, b: 2), 0.001);
29     }
30
31     @Test
32     public void testDividirPorCero() {
33         System.out.println(x: "Ejecutando testDividirPorCero...");
34         // Verifica que lance la excepción (Bug 2)
35         assertThrows(ArithmeticException.class, () -> {
36             calc.dividir(a: 10, b: 0);
37         });
38     }
39 }
```

Test Runner for Java

- CalculadoraTest \$(symbol-namespace)...
- testDividir() \$(symbol-class) Calculador...
 - Expected [5.0] but was [-5.0]
 - org.opentest4j.AssertionFailedError: expec...

9 older results

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

%TESTC 1 v2

%TSTTREE2,CalculadoraTest,true,1,false,1,CalculadoraTest,,[engine:junit-jupiter]/[class:CalculadoraTest]

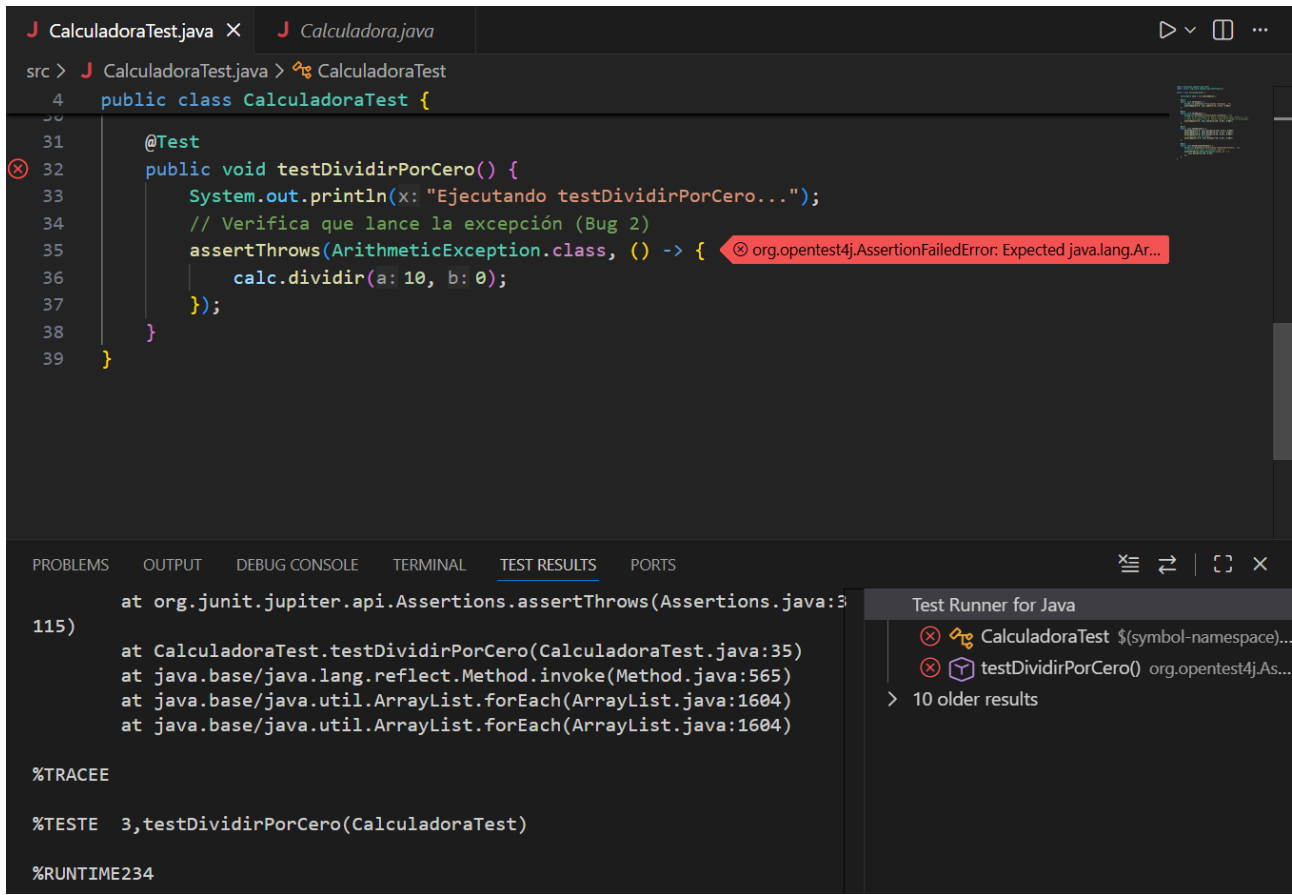
%TSTTREE3,testDividir(CalculadoraTest),false,1,false,2,testDividir(),,[engine:junit-jupiter]/[class:CalculadoraTest]/[method:testDividir()]

%TESTS 3,testDividir(CalculadoraTest)

%TESTE 3,testDividir(CalculadoraTest)

%RUNTIME194

4. **testDividirPorCero:** Implementa el caso CP-DIV-03 utilizando `assertThrows` para verificar que el sistema lance una `ArithmeticException` ante una división por cero.



```
src > J CalculadoraTest.java > CalculadoraTest
4 public class CalculadoraTest {
31
32     @Test
33     public void testDividirPorCero() {
34         System.out.println(x: "Ejecutando testDividirPorCero...");
35         // Verifica que lance la excepción (Bug 2)
36         assertThrows(ArithmeticException.class, () -> {
37             calc.dividir(a: 10, b: 0);
38         });
39     }
}
```

115) at org.junit.jupiter.api.Assertions.assertThrows(Assertions.java:3
at CalculadoraTest.testDividirPorCero(CalculadoraTest.java:35)
at java.base/java.lang.reflect.Method.invoke(Method.java:565)
at java.base/java.util.ArrayList.forEach(ArrayList.java:1604)
at java.base/java.util.ArrayList.forEach(ArrayList.java:1604)

%TRACEE

%TESTE 3,testDividirPorCero(CalculadoraTest)

%RUNTIME234

Test Runner for Java

- CalculadoraTest \$(symbol-namespace)...
- testDividirPorCero() org.opentest4j.As...

> 10 older results

Resultados de la Ejecución Inicial:

Tras ejecutar la batería de pruebas, se han obtenido los siguientes resultados (ver Capturas):

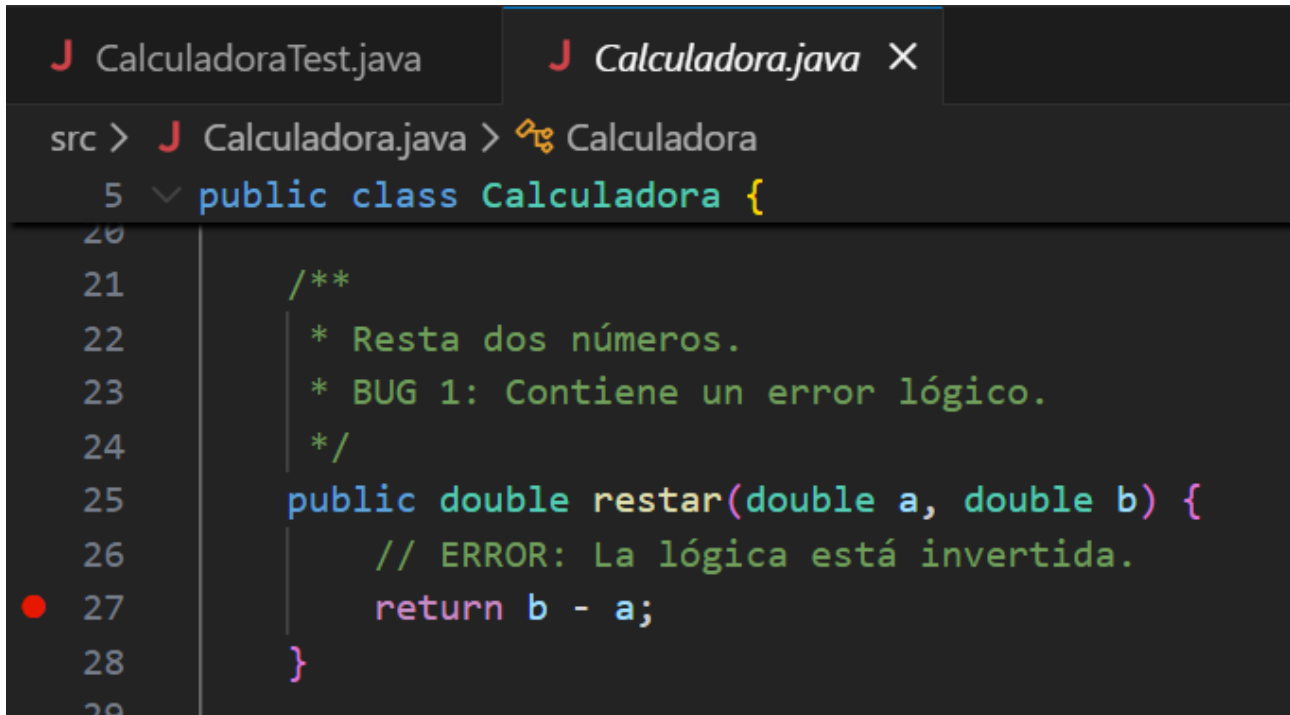
- **Fallo en testRestar:** La prueba ha fallado. JUnit reporta que se esperaba un valor de 5.0 pero se obtuvo -5.0. Esto confirma la existencia del Bug 1 (lógica invertida en la resta).
- **Fallo en testDividirPorCero:** La prueba ha fallado. Se esperaba que el código lanzara una `ArithmeticException`, pero no lo hizo (el método devolvió Infinity debido a la naturaleza de los tipos `double`). Esto confirma el Bug 2 (falta de control de excepciones).
- **testSumar y resto de divisiones:** Han pasado correctamente, validando la lógica base de esas operaciones.

Parte 3: Depuración e investigación del error

Tarea 4: Identificar y usar herramientas

Para investigar la causa del fallo en la prueba testRestar, se ha utilizado el depurador integrado en Visual Studio Code. El proceso seguido ha sido:

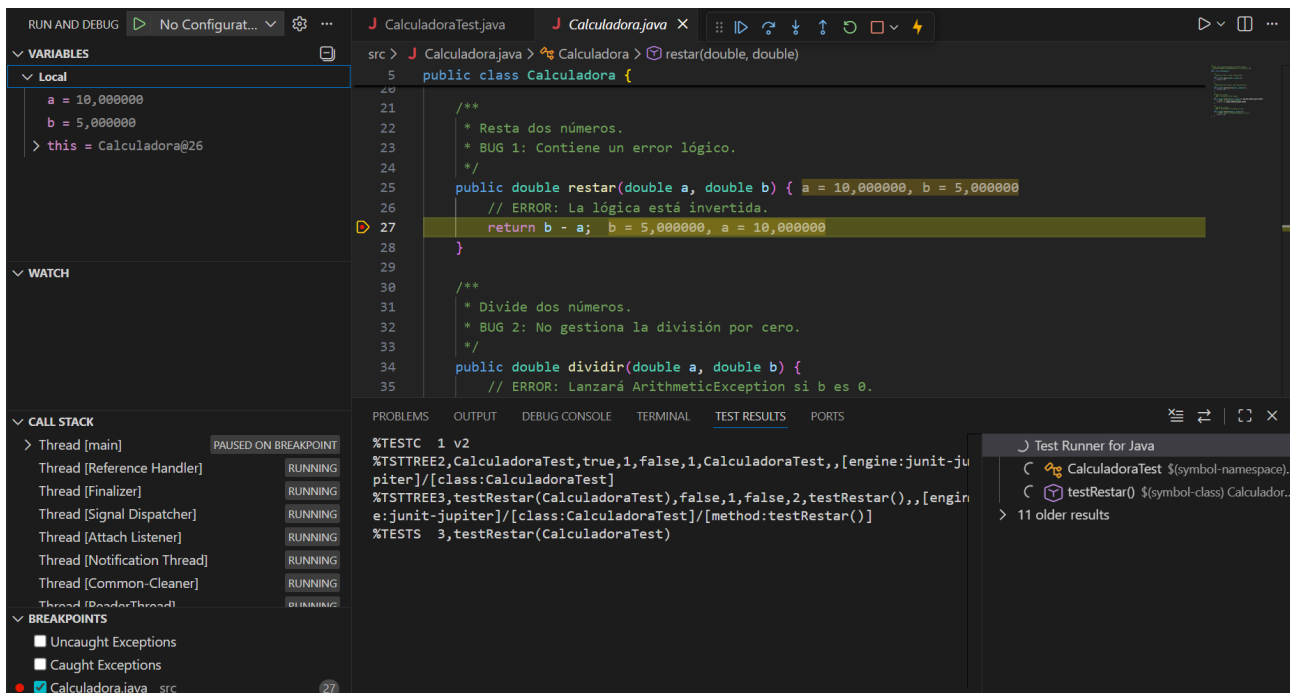
1. Definición del punto de ruptura (Breakpoint): Se ha colocado un breakpoint en la línea 27 de la clase Calculadora.java (instrucción return b - a;). Esto permite detener la ejecución justo antes de que se realice la operación defectuosa.



The screenshot shows the Visual Studio Code editor with two tabs: 'CalculadoraTest.java' and 'Calculadora.java'. The 'Calculadora.java' tab is active, showing the source code of the 'Calculadora' class. A breakpoint (red dot) is set on line 27, which contains the statement 'return b - a;'. The code includes comments in Spanish: 'Restar dos números.' and 'BUG 1: Contiene un error lógico.'.

```
src > J Calculadora.java > Calculadora
5  public class Calculadora {
20
21      /**
22       * Resta dos números.
23       * BUG 1: Contiene un error lógico.
24       */
25      public double restar(double a, double b) {
26          // ERROR: La lógica está invertida.
27          return b - a;
28      }
29
```

2. Ejecución paso a paso: Se ha lanzado el test testRestar en modo Debug. La ejecución se detuvo correctamente en la línea marcada.



The screenshot shows the Visual Studio Code interface in Debug mode. The 'RUN AND DEBUG' toolbar at the top indicates the application is paused. The 'VARIABLES' pane on the left shows local variables 'a = 10,000000' and 'b = 5,000000'. The 'WATCH' pane is empty. The 'CALL STACK' pane shows the current thread 'Thread [main]' is paused on the breakpoint. The 'PROBLEMS' pane shows no errors. The 'TEST RESULTS' pane shows the test 'testRestar' failed. The 'DEBUG CONSOLE' pane shows the test execution details.

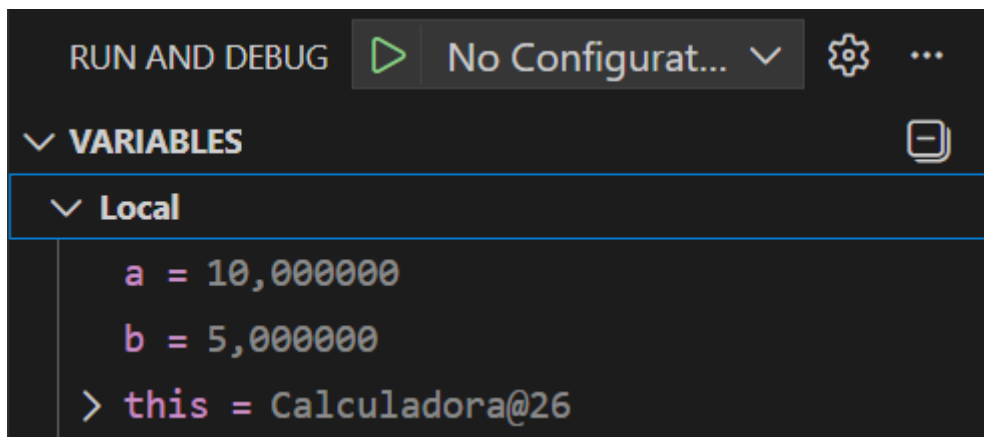
```
src > J Calculadora.java > Calculadora > restar(double, double)
5  public class Calculadora {
20
21      /**
22       * Resta dos números.
23       * BUG 1: Contiene un error lógico.
24       */
25      public double restar(double a, double b) {
26          // ERROR: La lógica está invertida.
27          return b - a;
28      }
29
30      /**
31       * Divide dos números.
32       * BUG 2: No gestiona la división por cero.
33       */
34      public double dividir(double a, double b) {
35          // ERROR: Lanzará ArithmeticException si b es 0.

```

TEST RESULTS

```
%TESTC 1 v2
%TSTTREE2,CalculadoraTest,true,1,false,1,CalculadoraTest,,[engine:junit-jupiter]/[class:CalculadoraTest]
%TSTTREE3,testRestar(CalculadoraTest),false,1,false,2,testRestar(),,[engine:junit-jupiter]/[class:CalculadoraTest]/[method:testRestar()]
%TESTS 3,testRestar(CalculadoraTest)
```

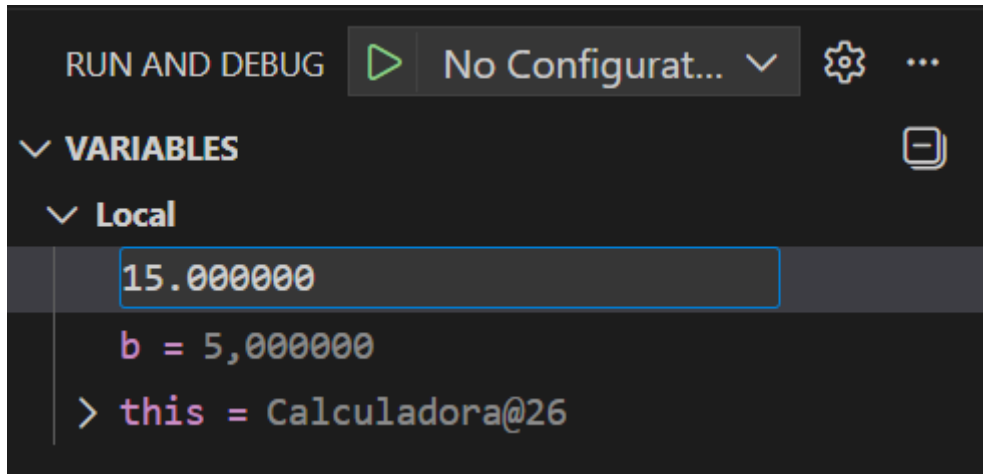
3. Inspección de variables: En el panel "Variables", se ha podido confirmar que los valores recibidos son correctos ($a = 10.0$, $b = 5.0$). Sin embargo, al observar el código detenido, se evidencia el error lógico: la operación escrita es $b - a$ ($5 - 10$), lo que explica el resultado incorrecto de -5.0 .



Tarea 5: Examinar y modificar en ejecución

Con el programa detenido en el punto de ruptura, se ha procedido a modificar el valor de las variables en memoria para observar cómo afecta al resultado final sin cambiar el código fuente.

1. Modificación de Variables: En la ventana "Variables", se modificó manualmente el valor de la variable `a`, cambiando de 10.0 a 15.0.



2. Verificación del Impacto: Tras reanudar la ejecución (botón "Continuar"), la prueba `testRestar` falló con un nuevo resultado: -10.0. Esto confirma que el programa utilizó el valor inyectado (15.0) y ejecutó la lógica errónea `b - a` ($5 - 15 = -10$).

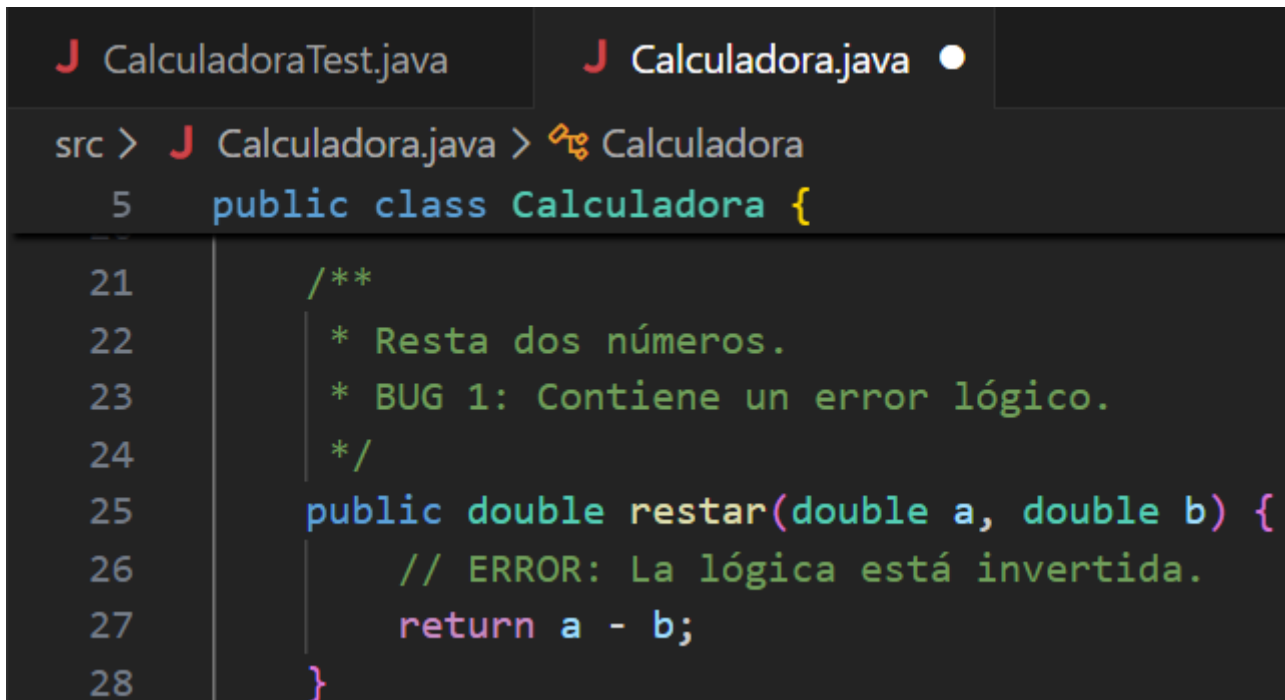
```
14      @Test
15      public void testRestar() {
16          System.out.println(x: "Ejecutando testRestar...");
17          // OJO: Si ya corregiste el bug en Calculadora.java (return a - b),
18          // este test pasará. Si no, fallará (que es lo normal al principio).
19          assertEquals(5.0, calc.restar(a: 10, b: 5), 0.001);
20      }
```

✗ -10.0 != 5.0

Solución de Bugs Detectados

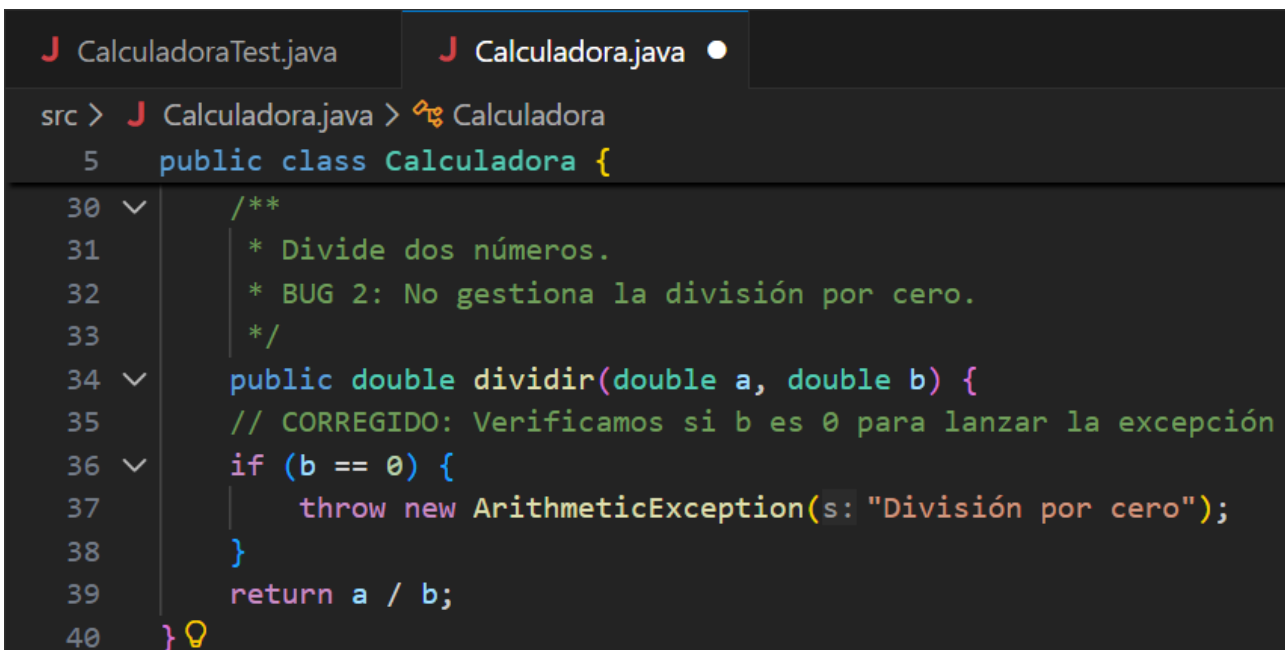
Tras identificar las causas de los fallos mediante la depuración, se han aplicado las siguientes correcciones en el código fuente (Calculadora.java):

1. **Corrección del Bug 1 (Resta):** Se ha corregido el error lógico en el método restar. La operación original `return b - a;` (invertida) se ha sustituido por `return a - b;` para obtener el resultado matemático correcto.



```
src > CalculadoraTest.java Calculadora.java
5 public class Calculadora {
21     /**
22      * Resta dos números.
23      * BUG 1: Contiene un error lógico.
24      */
25     public double restar(double a, double b) {
26         // ERROR: La lógica está invertida.
27         return a - b;
28     }
```

2. **Corrección del Bug 2 (División por Cero):** Dado que la división de números de punto flotante (double) en Java devuelve Infinity en lugar de lanzar una excepción, se ha implementado un control explícito. Se ha añadido una condicional `if (b == 0)` que lanza manualmente una `ArithmeticException` cuando el divisor es cero.

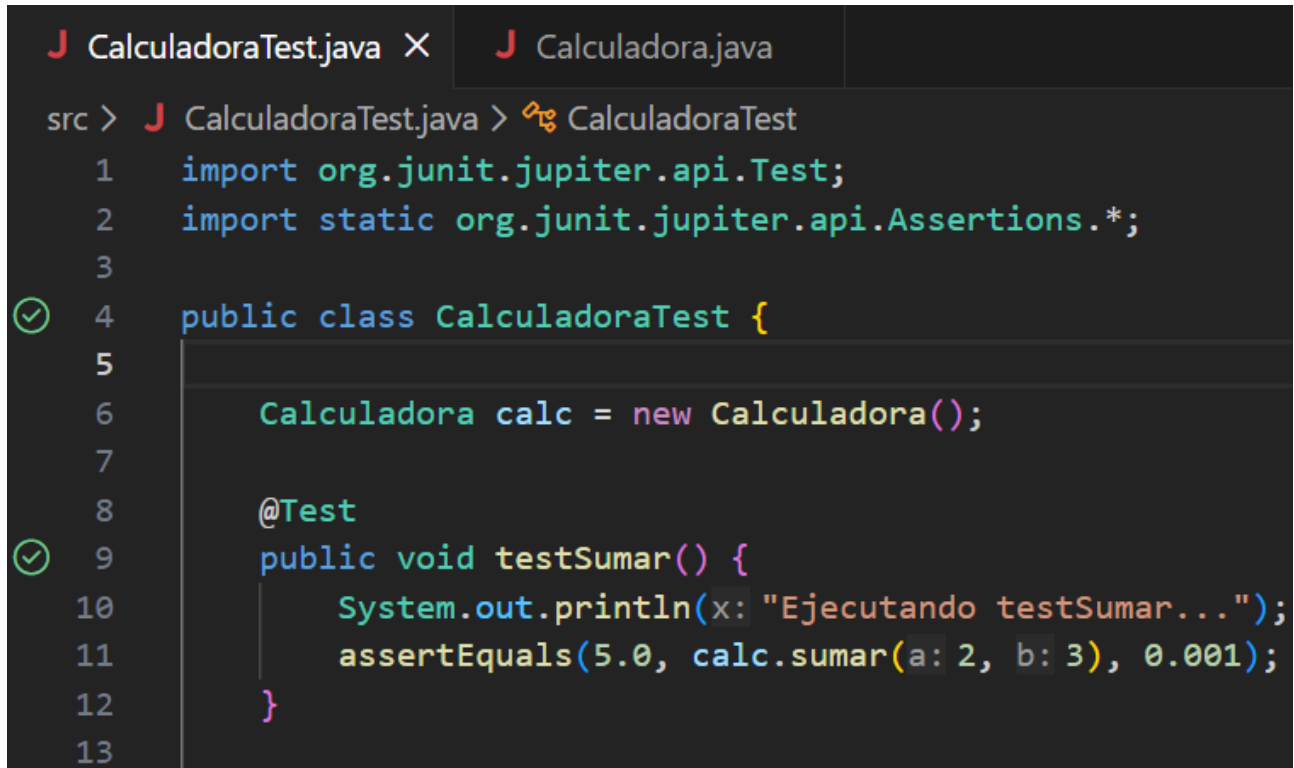


```
src > CalculadoraTest.java Calculadora.java
5 public class Calculadora {
30     /**
31      * Divide dos números.
32      * BUG 2: No gestiona la división por cero.
33      */
34     public double dividir(double a, double b) {
35         // CORREGIDO: Verificamos si b es 0 para lanzar la excepción
36         if (b == 0) {
37             throw new ArithmeticException(s: "División por cero");
38         }
39         return a / b;
40     }
```

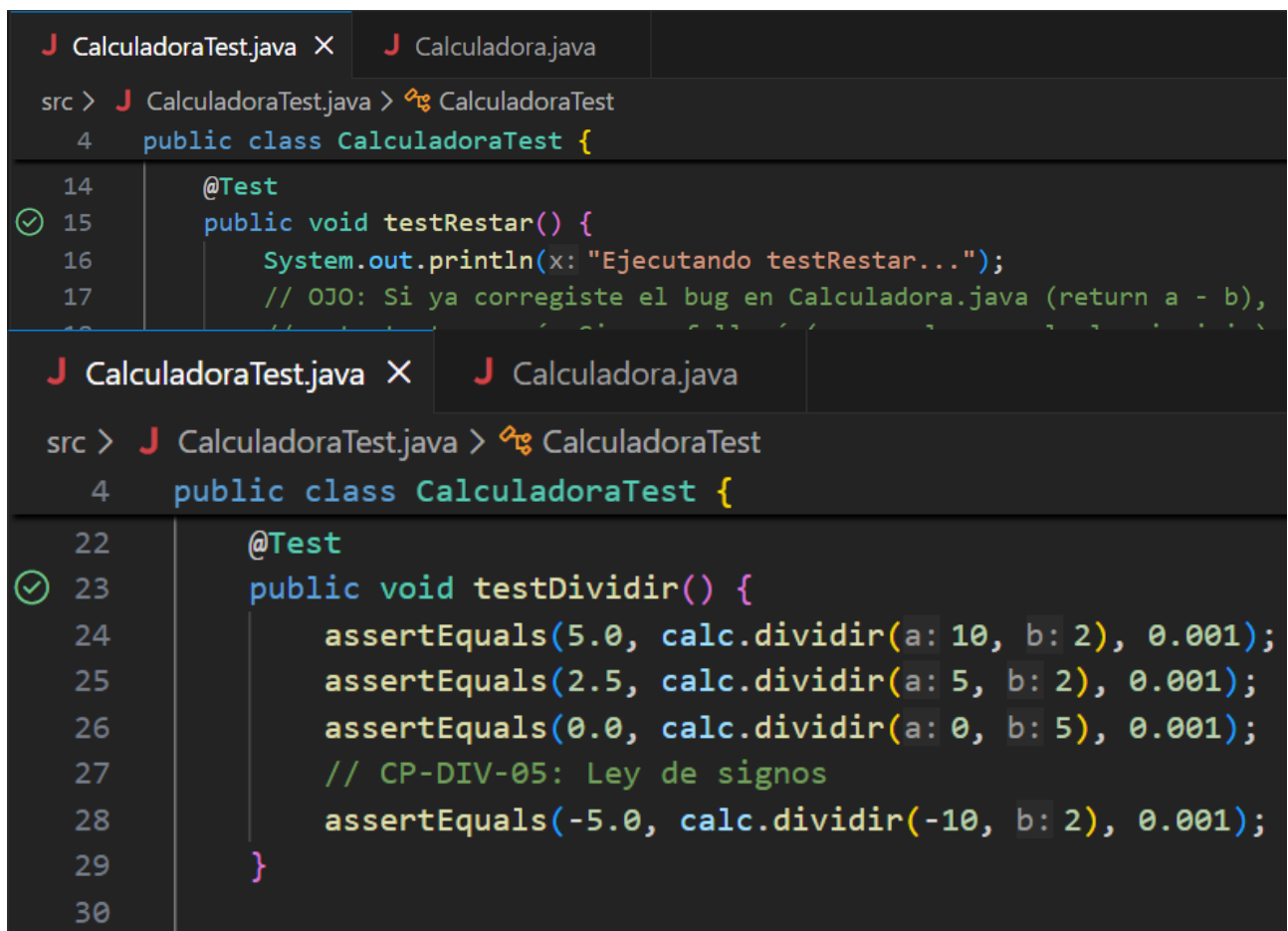
Verificación de Correcciones (Pruebas de Regresión)

Una vez aplicadas las correcciones en los métodos restar y dividir, se ha procedido a ejecutar nuevamente la batería completa de pruebas unitarias. Como se observa en la captura de resultados:

1. Todos los tests han pasado exitosamente.
2. testRestar: Ahora valida correctamente que $10 - 5 = 5$, confirmando que la inversión lógica ha sido solucionada.
3. testDividirPorCero: El test pasa porque el código ahora lanza explícitamente la `ArithmeticException` controlada, cumpliendo con el requisito de robustez ante errores.



```
src > CalculadoraTest.java > CalculadoraTest
1  import org.junit.jupiter.api.Test;
2  import static org.junit.jupiter.api.Assertions.*;
3
4  public class CalculadoraTest {
5
6      Calculadora calc = new Calculadora();
7
8      @Test
9      public void testSumar() {
10         System.out.println(x: "Ejecutando testSumar...");
11         assertEquals(5.0, calc.sumar(a: 2, b: 3), 0.001);
12     }
13
```



```
src > CalculadoraTest.java > CalculadoraTest
4  public class CalculadoraTest {
14
15      @Test
16      public void testRestar() {
17         System.out.println(x: "Ejecutando testRestar...");
18         // OJO: Si ya corregiste el bug en Calculadora.java (return a - b),
19         assertEquals(5.0, calc.restar(a: 10, b: 5), 0.001);
20     }
21
22     @Test
23     public void testDividir() {
24         assertEquals(5.0, calc.dividir(a: 10, b: 2), 0.001);
25         assertEquals(2.5, calc.dividir(a: 5, b: 2), 0.001);
26         assertEquals(0.0, calc.dividir(a: 0, b: 5), 0.001);
27         // CP-DIV-05: Ley de signos
28         assertEquals(-5.0, calc.dividir(-10, b: 2), 0.001);
29     }
30
```

```

31     @Test
32     public void testDividirPorCero() {
33         System.out.println(x: "Ejecutando testDividirPorCero...");
34         // Verifica que lance la excepción (Bug 2)
35         assertThrows(ArithmeticException.class, () -> {
36             calc.dividir(a: 10, b: 0);
37         });
38     }
39 }

```

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
TEST RESULTS
PORTS

e:junit-jupiter][[class:CalculadoraTest][[method:testRestar()]]
%TESTS 3,testSumar(CalculadoraTest)

%TESTE 3,testSumar(CalculadoraTest)

%TESTS 4,testDividirPorCero(CalculadoraTest)

%TESTE 4,testDividirPorCero(CalculadoraTest)

%TESTS 5,testDividir(CalculadoraTest)

%TESTE 5,testDividir(CalculadoraTest)

%TESTS 6,testRestar(CalculadoraTest)

%TESTE 6,testRestar(CalculadoraTest)

%RUNTIME172

Test Runner for Java

- ✓ CalculadoraTest \$(symbol-namespace)...
- ✓ testDividir() \$(symbol-class) Calculador...
- ✓ testDividirPorCero() \$(symbol-class) C...
- ✓ testRestar() \$(symbol-class) Calculador...
- ✓ testSumar() \$(symbol-class) Calculador...

> 12 older results

Parte 4: Aislamiento con dobles de prueba

Tarea 6: Uso de dobles de prueba (mocks)

Objetivo: Verificar el correcto funcionamiento de la clase GestorFacturas de forma aislada, asegurando que la lógica del cálculo del IVA es correcta independientemente de la implementación de Calculadora.

Implementación: Se ha utilizado la librería Mockito para crear un "doble" (Mock) de la dependencia Calculadora. El flujo de la prueba es el siguiente:

1. Mocking: Se crea una instancia simulada con `mock(Calculadora.class)`.
2. Stubbing (Entrenamiento): Se define el comportamiento esperado: cuando se llame a `multiplicar(100.0, 1.21)`, el mock devolverá 121.0 inmediatamente, sin ejecutar código real.
3. Verificación: Se comprueba con `assertEquals` que el `GestorFacturas` devuelve el valor esperado y con `verify` que realmente utilizó la calculadora para realizar la operación.

The screenshot shows the Visual Studio Code interface. The top editor pane displays the file `GestorFacturasTest.java` with the following code:

```
src > J GestorFacturasTest.java > GestorFacturasTest > testCalcularTotalFactura()
1  import org.junit.jupiter.api.Test;
2  import static org.junit.jupiter.api.Assertions.*;
3  import static org.mockito.Mockito.*;
4
5  public class GestorFacturasTest {
6
7      @Test
8      public void testCalcularTotalFactura() {
9          // 1. CREAR EL MOCK
10         Calculadora calcMock = mock(Calculadora.class);
11
12         // 2. ENTRENAR AL MOCK
13         // Cuando le pidan multiplicar 100 * 1.21, devuelve 121.0
14         when(calcMock.multiplicar(a: 100.0, b: 1.21)).thenReturn(121.0);
15
16         // 3. EJECUTAR LA PRUEBA
17         GestorFacturas gestor = new GestorFacturas();
18
19         // Le pasamos la calculadora falsa
```

The bottom editor pane shows the `TEST RESULTS` tab with the following output:

```
%TSTTREE2,GestorFacturasTest,true,1,false,1,GestorFacturasTest,,[engine:junit-jupiter]/[class:GestorFacturasTest]
%TSTTREE3,testCalcularTotalFactura(GestorFacturasTest),false,1,false,2,testCalcularTotalFactura(),,[engine:junit-jupiter]/[class:GestorFacturasTest]/[method:testCalcularTotalFactura()]
%TESTS 3,testCalcularTotalFactura(GestorFacturasTest)

%TESTE 3,testCalcularTotalFactura(GestorFacturasTest)

%RUNTIME1312
```

On the right side, the `Test Runner for Java` panel shows two test results, both marked as passed (green checkmarks):

- `GestorFacturasTest $(symbol-namespace) ...`
- `testCalcularTotalFactura() $(symbol-class) ...`

Below the test results, it indicates `> 6 older results`.

Anexo Técnico: Configuración del Entorno y Resolución de Problemas

Durante la implementación de las pruebas con Mockito, se detectaron conflictos de compatibilidad debido al uso de una versión de desarrollo reciente de Java (Java 25). Para lograr la ejecución exitosa de los tests, fue necesario realizar una configuración avanzada del entorno en Visual Studio Code:

Argumentos de la JVM (VM Args): Mockito utiliza internamente la librería Byte Buddy, la cual requiere permisos especiales para "inyectar" código en tiempo de ejecución en las versiones modernas de Java (Strong Encapsulation). Se configuró el archivo `.vscode/settings.json` con los siguientes argumentos:

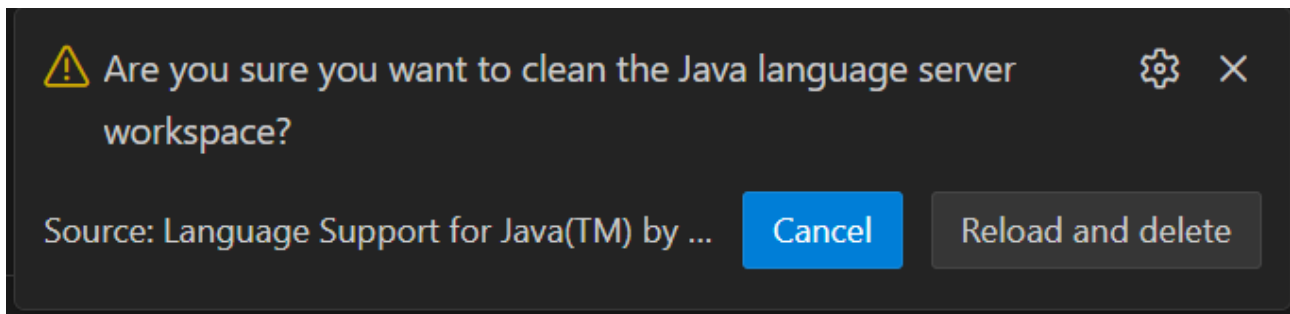
-XX:+EnableDynamicAgentLoading: Permite la carga dinámica de agentes (necesario para el mocking).

-Dnet.bytebuddy.experimental=true: Habilita el soporte experimental en Byte Buddy para trabajar con versiones de Java (como la 25) que aún no están oficialmente soportadas por la librería estándar.



```
.vscode > {} settings.json > ...
1  {
2    "java.project.referencedLibraries": [
3      "lib/**/*.jar"
4    ],
5    "java.test.config": {
6      "vmArgs": [
7        "-XX:+EnableDynamicAgentLoading",
8        "-Djdk.attach.allowAttachSelf=true",
9        "--add-opens=java.base/java.lang=ALL-UNNAMED",
10       "--add-opens=java.base/java.util=ALL-UNNAMED",
11        "-Dnet.bytebuddy.experimental=true"
12      ]
13    }
14  }
```

Limpieza del Workspace: Para asegurar que VS Code aplicara la nueva configuración de la máquina virtual, se procedió a eliminar las configuraciones cacheadas antiguas (borrado de .vscode y .classpath) y se forzó una limpieza del servidor de lenguaje (Java Clean: Restart and Delete).



Esta configuración ha permitido ejecutar la batería completa de pruebas en un entorno Java de última generación sin errores de instrumentación.

Durante la configuración inicial del proyecto en Visual Studio Code, se presentaron bloqueos técnicos ajenos al código fuente que impedían la ejecución de las pruebas. A continuación se detallan las incidencias y las soluciones aplicadas:

Conflictos de Codificación en Rutas (Encoding):

- **Incidencia:** Las librerías de Java (específicamente el agente de Mockito) fallaban al intentar cargar archivos desde una ruta que contenía caracteres especiales (tildes), concretamente en el directorio de usuario "Daniel González".
- **Solución:** Se procedió a sanear la ruta del sistema de archivos, renombrando el directorio a "Gonzalez" (sin tilde) para garantizar la compatibilidad con la codificación de rutas de la JVM.

The screenshot shows a VS Code editor window with a file named `GestorFacturasTest.java`. The code is a JUnit test for a `GestorFacturas` class. It uses Mockito to mock a `Calculadora` class. The test method `testCalcularTotalFactura()` sets up a mock that returns 121.0 for the multiplication of 100.0 and 1.21. The test then creates a `GestorFacturas` instance and calls its `calcularTotalFactura()` method. A red error banner at the top of the code editor indicates a `java.lang.IllegalStateException: Could not` error. The bottom of the screen shows the 'DEBUG CONSOLE' tab with the message: 'Error opening zip file or JAR manifest missing: C:\Users\Usuario\Desktop\Mis Cosas\Grado Superior\1º GS DAM\Asignaturas\Daniel González. Entornos de Desarrollo\Trabajo RA3 Final\lib\byte-buddy-agent-1.14.12.jar'.

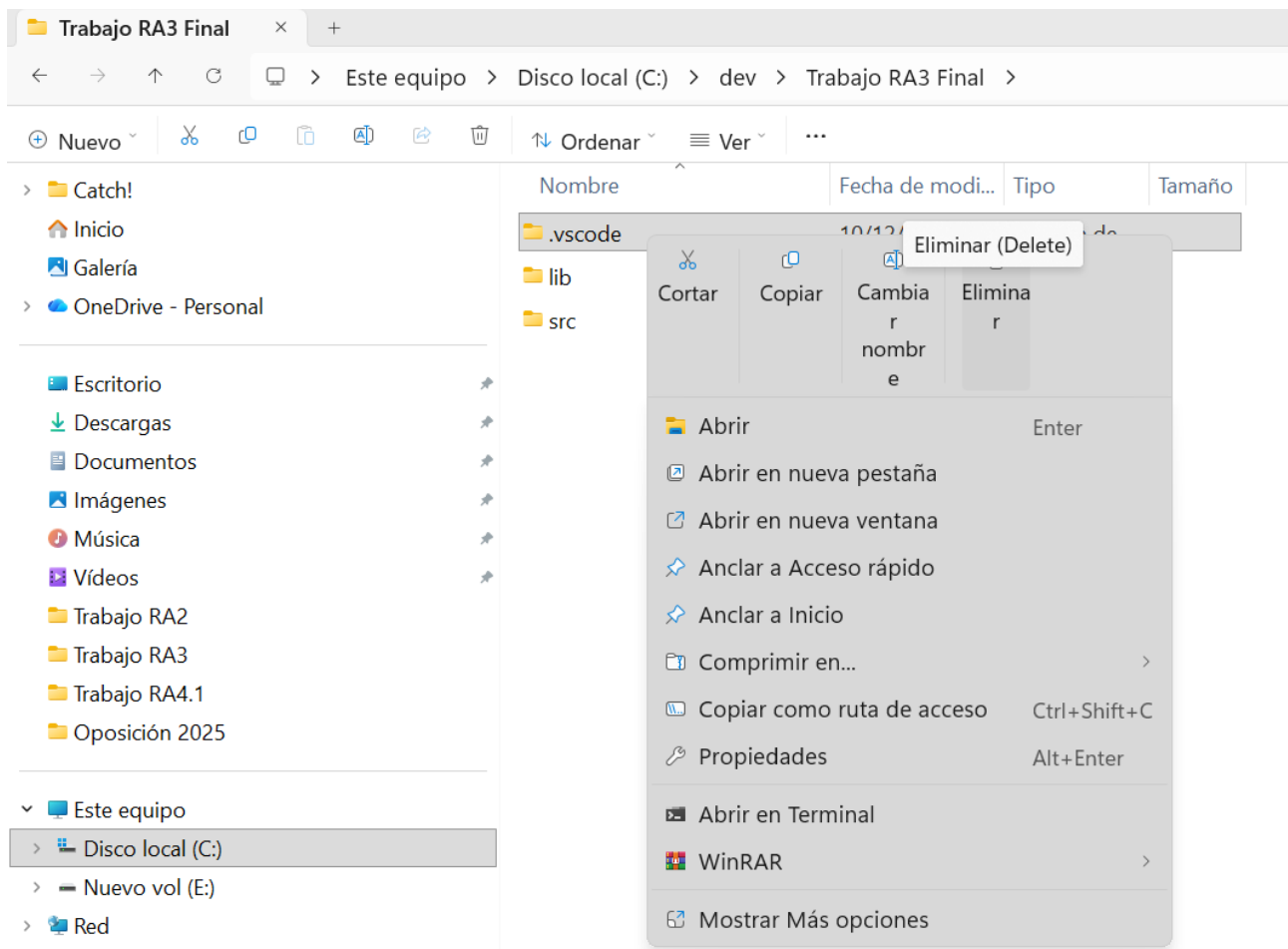
```
src > J GestorFacturasTest.java > GestorFacturasTest > testCalcularTotalFactura()
1  import org.junit.jupiter.api.Test;
2  import static org.junit.jupiter.api.Assertions.*;
3  import static org.mockito.Mockito.*;
4
5  public class GestorFacturasTest {
6
7      @Test
8      public void testCalcularTotalFactura() {
9          // 1. CREAR EL MOCK
10         Calculadora calcMock = mock(classToMock: Calculadora.class);
11
12         // 2. ENTRENAR AL MOCK
13         // Cuando le pidan multiplicar 100 * 1.21, devuelve 121.0
14         when(calcMock.multiplicar(a: 100.0, b: 1.21)).thenReturn(value: 121.0);
15
16         // 3. EJECUTAR LA PRUEBA
17         GestorFacturas gestor = new GestorFacturas();
18
19         // Le pasamos la calculadora falsa
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS Filter (e.g. text, !exclude, \esca... Q [] X

Error opening zip file or JAR manifest missing: C:\Users\Usuario\Desktop\Mis Cosas\Grado Superior\1º GS DAM\Asignaturas\Daniel González. Entornos de Desarrollo\Trabajo RA3 Final\lib\byte-buddy-agent-1.14.12.jar

Longitud de Ruta y Caché del IDE:

- **Incidencia:** A pesar de corregir la tilde, la anidación profunda de carpetas provocaba errores de lectura ("Error opening zip file"). Además, Visual Studio Code mantenía cacheadas las referencias a la ubicación antigua a través de archivos ocultos de metadatos (.classpath, .project).
- **Solución:**
 1. Se realizó una migración completa del proyecto a una ruta raíz corta y limpia: C:/dev/TrabajoRA3.
 2. Se eliminaron manualmente la carpeta .vscode y los archivos de configuración ocultos para forzar al IDE a regenerar el entorno desde cero en la nueva ubicación.



Incompatibilidad de Versión (Java 25 vs Mockito):

- **Incidencia:** Al utilizar Java 25 (versión Early Access), la librería de mocking (Byte Buddy) lanzaba una excepción `IllegalArgumentException` al no reconocer la versión de la JVM como soportada.
- **Solución:** Se generó un nuevo archivo `.vscode/settings.json` configurando argumentos específicos para la Máquina Virtual (VM Args). Se añadieron las banderas `-XX:+EnableDynamicAgentLoading` y, crucialmente, `-Dnet.bytebuddy.experimental=true`. Esta última fuerza a la librería a operar en modo experimental, permitiendo la ejecución exitosa de los tests en esta versión moderna de Java.

J GestorFacturasTest.java X {} settings.json

src > J GestorFacturasTest.java

```
1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3 import static org.mockito.Mockito.*;
4
5 public class GestorFacturasTest {
6
7     @Test
8     public void testCalcularTotalFactura() {
9         // 1. CREAR EL MOCK
10        Calculadora calcMock = mock(Calculadora.class);
11
12        // 2. ENTRENAR AL MOCK
13        // Cuando le pidan multiplicar 100 * 1.21, devuelve 121.0
14        when(calcMock.multiplicar(a: 100.0, b: 1.21)).thenReturn(121.0);
15
16        // 3. EJECUTAR LA PRUEBA
17        GestorFacturas gestor = new GestorFacturas();
18
19        // Le pasamos la calculadora falsa
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Filter (e.g. text, !exclude, \esca... Q X

```
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::objectFieldOffset has been called by net.bytebuddy.dynamic.loading.ClassInjector$UsingUnsafe$Dispatcher$CreationAction (file:/C:/dev/Trabajo%20RA3%20Final/lib/byte-buddy-1.14.12.jar)
WARNING: Please consider reporting this to the maintainers of class net.bytebuddy.dynamic.loading.ClassInjector$UsingUnsafe$Dispatcher$CreationAction
WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future release
```